Habidy: Designing a Habit & Identity System for Students

DS 593 LLM — Boston University *Eric Alberg and Hunter Mckeon*

Introduction

Our project was the app Habidy, which is an identity-focused, agentic habit-building tool focused on helping users develop daily habits that help shape them into who they want to be. The problem it addresses is that users face a lack of direction rather than a lack of productivity. Habidy solves this problem by drawing recommendations from James Clear's *Atomic Habits*, using two AI agents to understand who the user is and build habits specifically around the identity they want to embody. We used Anthropic API, Voyage AI, Supabase, Vercel, Langsmith, RAGAS, and Next.js to design, implement, and evaluate our system.

Approach

We utilized Next.js 16 alongside the App Router, TypeScript, and Tailwind CSS v4 to design our interface, hosting the final build on Vercel while leveraging Supabase for Postgres database management and authentication. Our system integrates AI capabilities via the Anthropic API, employing a model-agnostic abstraction layer, which is in the repo (lib/claude.ts), that allows any model to be swapped in the environment variable without having to rewrite any of the application code. A user's experience runs through our two agents: the investigation agent, which is focused on collecting user information, and the Architect agent, which generates habit options based on what the Investigative agent found. Having two separate agents was a deliberate architectural design, and it held up throughout the project.

RAG Pipeline

To ground the agents' recommendations in the actual text of *Atomic Habits*, we built a Retrieval-Augmented Generation (RAG) pipeline using Voyage AI embeddings and Supabase's pgvector extension. The source corpus was the full PDF of *Atomic Habits* by James Clear, which was processed using a standalone TypeScript ingestion script. Text extraction was done using pdftotext, then we split it into 1500-character chunks with a 150-character overlap, and filtered out any chunk under 100 characters. Each chunk was embedded using the voyage-3 model to get 1024-dimensional vectors, and we stored those in a Supabase rag_document. For embedding, we had to do it in batches of 10 with 2-second delays due to API rate limits. When the user runs the query, the user's input is embedded with the same model and passed to a PostgreSQL procedure, match_documents, which returns the 5 most similar chunks.

Investigation Agent

Its persona is that of an empathetic AI mentor: it asks exactly one question per turn, always references the user's stated identity goal by name, and never suggests habits — that is strictly the Architect's responsibility. The agent operates in one of three modes selected before the conversation begins: **Guided** (5 turns, structured topic sequence), **Deep** (15 turns, exploring identity, environment, blockers, and motivation in thematic clusters), and **Quick Habit** (a 2-minute form for users who already know what they want, which bypasses the investigation entirely and feeds directly to Architect).

System prompts are constructed dynamically from a rich context object that includes the user's identity statement, onboarding questionnaire answers (sleep patterns, stress level, existing anchor habits, biggest time wasters, location, and goal clarity), and an AI-generated running profile summary that persists and updates across sessions. Across the conversation, the agent internally tracks seven fields: who the user wants to become, the actions that person takes, what makes those actions attractive, environment, cue, a two-minute version of the target behavior, and barriers.

When the agent is ready to close the conversation, it provides a plain-text recap for the user followed by a structured JSON summary marker. The route handler detects this marker, splits the recap from the JSON, enriches it with onboarding data, and writes the result to the conversation_memory table. If the hard turn cap is reached without a natural summary, the route bypasses the LLM entirely and calls a forced summary prompt with the full conversation transcript to extract the structured output.

Architect Agent

The Architect reads the Investigator agent’s summary from conversation_memory and generates exactly five personalized habit options. Every habit follows a rigid format enforced in the system prompt: an identity label in the form "I am a ___", a habit name, a cue following the structure "After I [existing routine], I will [new behavior] at [location]", a two-minute version, and a category from one of six fixed strings. The output is parsed from the model's response using regex and JSON-parsed into typed habit objects. All five habits are inserted into a proposed_habits table; the user selects up to two from a swipe carousel in the UI, and the remaining three surface after a 7-day streak milestone.

Evaluation

RAG Evaluation

The RAG pipeline was evaluated using the RAGAS Python library, a framework specifically designed for assessing retrieval-augmented generation systems. Two metrics were measured: faithfulness, which assesses whether generated answers are grounded in the retrieved source material, and answer relevancy, which assesses whether answers actually address the question asked. A test set of 10 questions was constructed covering core *Atomic Habits* concepts — the habit loop, the two-minute rule, identity-based habits, implementation intentions, habit stacking, environment design, reward systems, and the 1% better principle.

Each question was answered twice: once by Claude Haiku with no context (the baseline, operating purely from general training knowledge), and once with the top 5 retrieved chunks from the rag_documents table injected as context. RAGAS used Claude Haiku as the judge LLM and Voyage AI voyage-3 embeddings for embedding-based scoring — the same models used in the pipeline itself, ensuring consistency.

Agent Quality Evaluation

The agent evaluation had two phases. The first phase was gathering the right material and context so that agents could ask the right questions to users. This information was gathered in user interviews from test users. The second phase was built using LangChain, where we used different models to test the prompts once they were built out: Claude Haiku, Claude Sonnet, GPT-4o-mini, and GPT-4o. Claude Haiku was the dummy LLM that simulated users across three persona types — expressive, moderate, and uninterested — while each candidate model played the Identity Gatherer agent in guided mode for up to 5 turns. The resulting conversations and summaries were passed to the Architect agent, and a judge LLM (Claude Sonnet 4.6) scored each run across five criteria per agent. The full evaluation ran 12 investigator sessions and 12 architect sessions — 24 LangSmith spans total — in batches of 3 with a 3-second delay. All runs were traced to a LangSmith project (Habidy-Prompt-Eval) for review.

The Investigative agent was scored on: question specificity (are questions tied to the user's specific goal?), atomic habits coverage (are cue, environment, time, energy, and two-minute version all surfaced?), question sharpness (are questions concise and direct?), vague user handling (does the agent probe deeper when the user is unhelpful?), and efficiency (is a summary produced within the turn budget?). The Architect was scored on: identity alignment, habit specificity, information utilization, journey display, and habit variety.

Results and Discussion

RAGAS Results

Metric	No RAG (Baseline)	RAG
Faithfulness	0.000	0.797
Answer Relevancy	0.581	0.555

Faithfulness jumped from 0.000 to 0.797. The baseline score of 0.000 indicates that without retrieved context, none of Claude's generated claims are traceable to a source document — the model is drawing entirely from training data. With RAG, approximately 80% of claims are directly supported by retrieved passages from *Atomic Habits*. Answer relevancy decreased slightly from 0.581 to 0.555, a difference of 0.026. This is attributable to cases where retrieved chunks introduced off-topic context — fixed-size character chunking occasionally surfaces passages that are adjacent but not directly relevant to the query, pulling answers marginally off-center.

LangChain Agent Evaluation Results

Criterion	Haiku	Sonnet	GPT-4o-mini	GPT-4o
Question Specificity	0.79	0.80	0.52	0.57
Atomic Habits Coverage	0.66	0.66	0.43	0.52
Question Sharpness	0.73	0.75	0.43	0.50
Vague User Handling	0.60	0.60	0.30	0.30
Efficiency	1.00	1.00	1.00	1.00
Average	**0.76**	**0.76**	**0.54**	**0.58**

Criterion	Haiku	Sonnet	GPT-4o-mini	GPT-4o
Identity Alignment	0.76	0.88	0.62	0.55
Habit Specificity	0.61	0.83	0.58	0.66
Information Utilization	0.72	0.88	0.47	0.47
Journey Display	0.64	0.78	0.47	0.47
Habit Variety	0.64	0.79	0.58	0.58
Average	**0.67**	**0.83**	**0.54**	**0.54**

Persona Breakdown (avg across all agents)

Model	Expressive	Moderate	Uninterested
Haiku	0.76	0.76	0.63
Sonnet	0.81	0.77	0.82
GPT-4o-mini	0.56	0.53	0.52
GPT-4o	0.58	0.56	0.55

Sonnet was the clear winner overall, averaging 0.80 across both agents and proving particularly strong with uninterested users (0.82 persona average) — the hardest persona type to handle well. Notably, Haiku matched Sonnet on the Constellation agent (0.76) and was the most efficient model across all runs, completing summaries within the turn budget every single time (1.00 efficiency). However, a revealing edge case emerged: when Haiku faced an uninterested user on the Architect stage, it failed to generate any habits at all — a habits count of 0 — highlighting that Haiku's strength is efficiency, not resilience. Sonnet's ability to synthesize meaningful habits from a limited context is a genuine differentiator and the

primary reason we selected it as the production model. This selection was validated both by the automated evaluation scores and by us reading the actual conversations directly — the Sonnet-generated habits were consistently more specific, more grounded in the user's stated identity, and more actionable than those produced by any other model.

Both GPT-4o-mini and GPT-4o underperformed significantly relative to the Claude models on nearly every criterion, particularly on vague user handling (0.30 for both). This suggests that Claude's conversational judgment is a meaningful advantage for this kind of nuanced, person-first interaction, where the model must do real work to extract useful information from someone who isn't cooperating. The two weakest criteria across all models were vague user handling (average 0.45) and atomic habits coverage (average 0.57) — both directly tied to the quality of habits Architect can generate downstream, making these the highest-priority targets for prompt improvement.

Ethics and Limitations

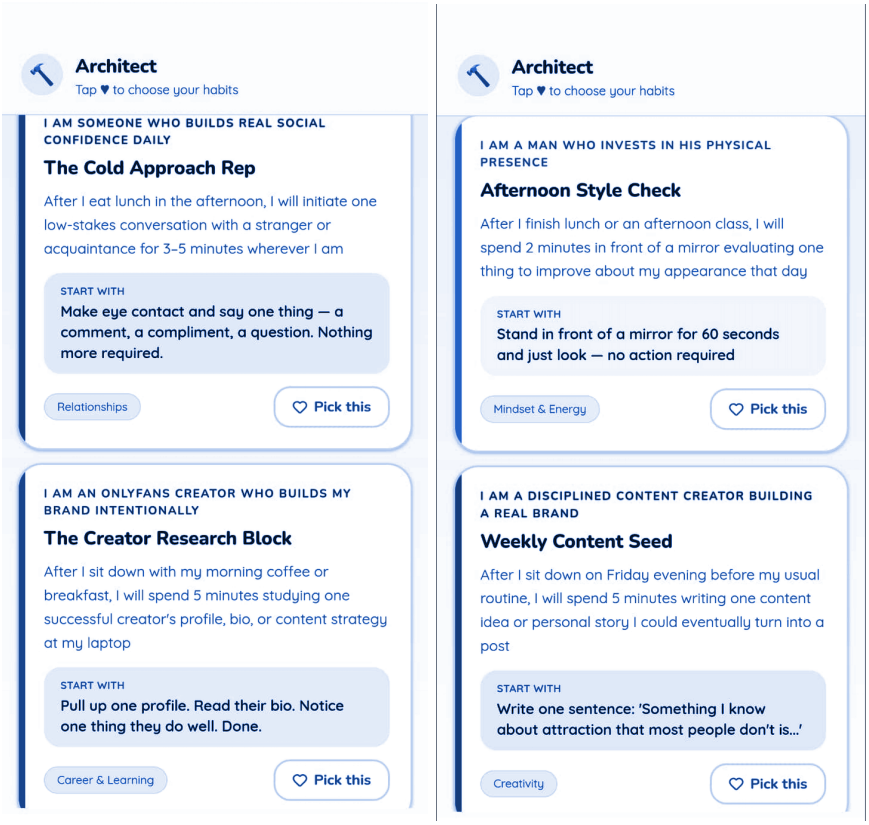
Habidy handles personal user information such as name and date of birth, and optionally, user location and calendar information. We store the data in Supabase, so in theory, the user data should be safe, but our tool uses a closed LLM (Anthropic and optionally OpenAI). This is an ethical issue because we wouldn’t have full control over the information that a user hands over to our app. For the sake of the project timelines, we used closed models, but our team has discussed wanting to move over to an open model where we could set more strict parameters and ensure that user data doesn’t go to any third parties.

The target user population — college students — deserves particular care. Students may be in emotionally difficult periods: identity uncertainty, academic pressure, and social anxiety. Framing the app around "who you want to become" is psychologically powerful, but carries risk. If the agent handles this framing poorly, it could inadvertently reinforce a sense of inadequacy — the feeling that the user is not yet the person they want to be, and that this is a problem. The app's design principle of "warm, not clinical" attempts to mitigate this, but agent prompts must be continuously evaluated for language that could feel judgmental or discouraging. We also noted that while guardrails were built into the agent prompts, there were cases where the AI was genuinely helpful toward goals that were not obviously positive — identity framings that could be seen as socially problematic depending on implementation. Claude's built-in safety guardrails handle the clearest cases, but the line between a coaching-oriented identity goal and a potentially harmful one is not always obvious, and open-ended identity elicitation creates real surface area for misuse that the current prompts do not fully address. Input sanitization and stricter validation on agent inputs are identified as near-term engineering priorities.

On user privacy and transparency, the app's design principles explicitly require that every AI involvement be made clear to the user. However, users may not fully understand that conversations are stored in a conversation_memory table and used to build a running profile summary that informs future sessions. Clearer in-app disclosure of this behavior is necessary and planned for future work.

Our last major ethical concern is people exploiting our app for negative or illegal behaviors. It was built with the idea that people want to build good habits and break bad ones, but we did not consider users trying to build negative habits. Should we allow users to do this? This is something we will have to

consider moving forward, because how do we determine negative or “wrong” when we can’t account for every user’s background or motivations? For this example, we tested whether it could provide suggestions on habits to become an OnlyFans creator, and the app completely obliged. This is where this would be a major ethical challenge, because whether or not this would be allowed would probably need to consider state legislation, the user’s age, and deciding whether we as creators can effectively and equitably determine what habits and suggestions our app should be allowed to output.



Future Work

The most immediate next step is connecting the RAG pipeline to the live agent calls. The infrastructure is fully built and evaluated — it just needs to be wired in. Beyond that, we want to experiment with semantic chunking rather than fixed character counts and test higher similarity thresholds to reduce the off-topic retrieval that caused the slight drop in answer relevancy. We're also thinking about expanding the RAG corpus beyond *Atomic Habits* to other texts like *The Power of Habit* and *Tiny Habits* to give the agents a broader knowledge base to draw from.

On evaluation, we'd like to scale the RAGAS test set to 50 or more questions to get more statistically reliable results, and add human evaluation to reduce the bias that comes with LLM-as-judge scoring. The LangChain evaluation currently only covers guided mode, so running it against standard and deep modes is also on the list.

For the agents themselves, we're planning a pattern-recognition agent — a longer conversation designed to surface the patterns in a user's life that are blocking their goals: distractions, stress-driven behaviors, bad habits. This agent wouldn't build habits directly but would feed its findings into the Identity Gatherer and Architect pipeline for richer, more personalized output. We're also considering a voice interaction layer so users can check in conversationally rather than through text. An analytics page showing habit completion trends and streak history over time is planned as well, which directly addresses one of the biggest pain points we saw in user research: people quitting when they can't see progress.

The bigger vision is for Habidy to eventually pull in data from Google Calendar, Notion, and other tools to build a complete picture of how a user actually spends their time. The goal is for the app to function as a habit builder, task manager, and identity coach simultaneously, getting smarter the longer it's used. The infrastructure we built in this stage: the agent pipeline, RAG layer, Supabase context architecture, and evaluation framework, gives us a solid foundation to build toward that.

Acknowledgements & Team Contributions

https://github.com/aericdahlberg/Habidy.

Hunter (Frontend) designed and implemented all screen layouts and UI across the application: the six-screen onboarding flow, mode selection, the Constellation and Architect chat pages, dashboard, explore, social, and profile screens. He built the key visual components, including SwipeCheckIn (a Framer Motion swipe card stack for daily habit check-in), HabitCard (swipe gestures and streak dot visualization), the Embla carousel for habit selection in the Architect screen, and the five-tab BottomNav. Tailwind CSS v4 styling was applied throughout, including the gradient themes per screen — teal and purple for agents, amber and teal for the welcome screen, and flat off-white for the core app. The full shadcn/ui component library (49 components) and Framer Motion animations were integrated across the app. He built the full RAG pipeline — the PDF ingestion script, Voyage AI embedding integration, the Supabase rag_documents schema, the match_documents stored procedure, and the lib/rag.ts retrieval layer.

Eric (Backend & Agents) designed and implemented the Constellation and Architect agent system prompts and all associated prompt-builder functions. He implemented the model-agnostic LLM abstraction (lib/claude.ts), the logging wrapper (lib/agentGuard.ts), and the LangSmith tracing integration, as well as all API routes including the agent endpoints, habits, memory, and explore. He designed the full Supabase schema and wrote all database migrations, and built and ran both the RAGAS evaluation script and the LangChain prompt and model comparison evaluation.

Course: DS 593 LLM — Boston University.

Acknowledgements: The knowledge base powering Habidy's RAG pipeline is James Clear's *Atomic Habits*. The system prompt design and agent personas draw heavily on the psychological framework. Clear outlines in that work.